Group Cyan

WeVoteLive

Test Plan Document

SE 3313B:
Section 001

Date: 04/10/2025

TABLE OF CONTENTS

1.0 INTRODUCTION

Purpose

This Test Plan Document outlines the strategy and activities for testing the multi-user, multi-transaction C++ server. The primary goal is to ensure that the server complies with the project expectations, supports multiple concurrent transactions, manages user sessions, and remains scalable underload. This document also contains an addendum briefly covering how the front-end UI was tested.

Scope

The document covers:

- **Unit Testing:** Validation of individual components and functions.
- **Functional Testing:** Verification that the server behaves as intended per requirements.
- **Performance Testing:** Evaluation of the server's behavior under load (concurrent connections, high transaction volume).

Definitions and Acronyms

- ACID: Atomicity, Consistency, Isolation, Durability.
- GUI: Graphical User Interface.
- CI/CD: Continuous Integration / Continuous Deployment.
- TDD: Test-Driven Development.
- GDB: GNU Debugger.
- MSYS2/MinGW: Development tools and environments on Windows.

2.0 Test Objectives

- Verify that all server modules (network communication, transaction management, session handling) function as intended.
- Validate adherence to ACID properties within transactions.
- Test concurrency handling: multiple simultaneous user transactions should run reliably.
- Assess performance under sustained load and high transaction rates.
- Ensure that all requirements specified in the Project Expectations document are met.

3.0 Test Items

The following items are within the test scope:

- **Network Communication Module:**
  - Protocol handling
  - Connection setup/teardown
- **Transaction Management Module:**
  - Begin, commit, and rollback functionalities
  - Consistency and isolation between transactions
- **User/Session Management Module:**
  - Session timeout handling
- **Concurrency Control:**

    - Multi-threading or multi-processing capabilities
- **Error Handling and Logging:**
    - Robustness under erroneous inputs

4.0 Testing Strategy

Unit Testing

- **Objective**: To validate each individual function and module in isolation.
- **Scope:**
    - Core transaction logic
    - Helper functions and utility libraries
    - Database interaction (if applicable) or in-memory data management
- **Approach:**
    - Write unit tests using C++.
    - Mock external dependencies (e.g., network I/O, file operations).
    - Execute tests upon any changes made.

Functional Testing

- **Objective:** To ensure that the server meets all specified functional requirements.
- **Scope**:
    - End-to-end communication between client and server.
    - Functionality of transaction processing and ACID compliance.
    - Host room creation, poll creation, participant joining and voting features, etc.
- **Approach**:
    - Use of manual testing methods.
    - Simulate a realistic sequence of operations, including multi-user scenarios.
    - Validate server responses against expected outcomes.

Performance Testing

- **Objective:** To evaluate the server's performance underload and confirm it can handle concurrent transactions.
- **Scope:**
    - Stress testing under high user and transaction volumes.
    - Load testing to determine if multiple concurrent connections degrade performance.
- **Approach:**
    - Performing real load scenarios upon the server.
    - Monitor metrics: response time, throughput, CPU and memory utilization.
    - Test duration: sustained load tests over extended periods to identify potential resource leaks.

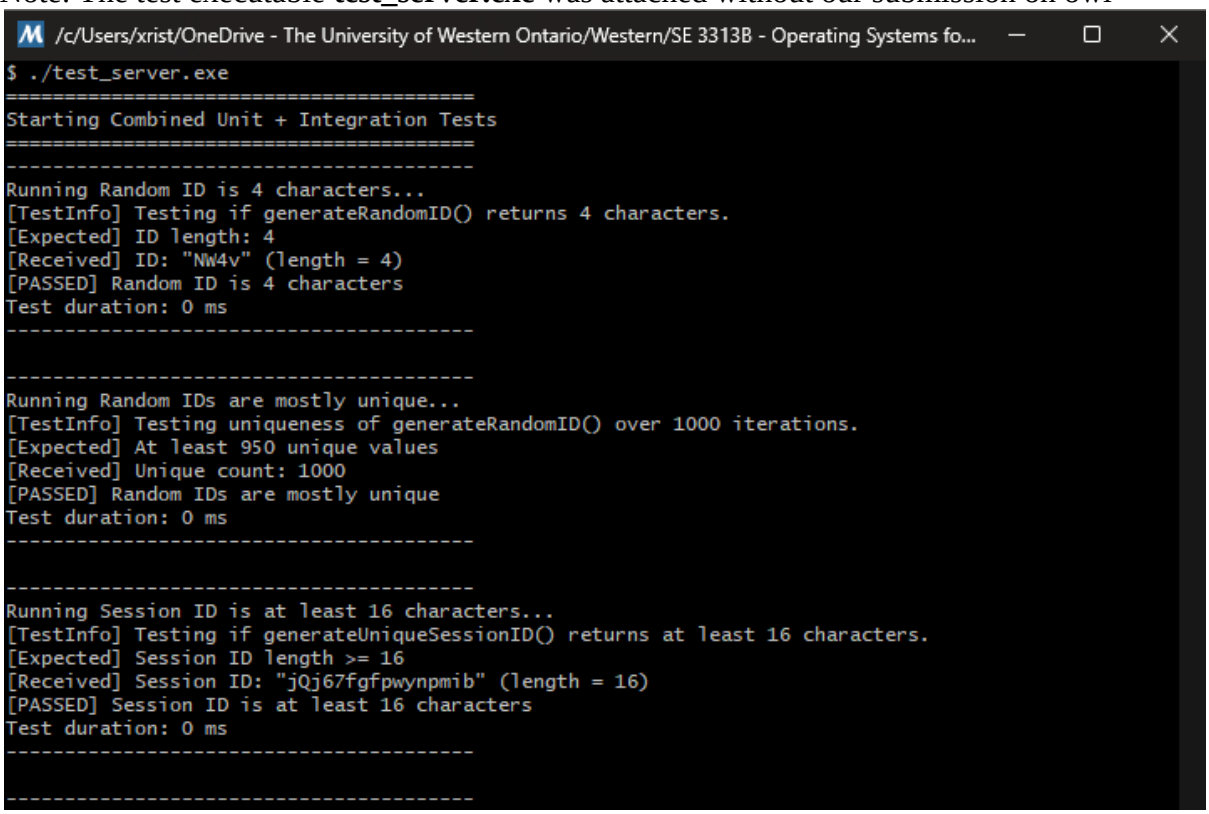
5.0 Test Environment

- Hardware:
  - Unit Tests and Performance Tests: Server deployed on local Windows 10/11 machine
  - Functional Tests: Server deployed on Linux environment
  - Client machines running various OS (Windows/Linux/Mac)
- Software:
  - Server: C++ built using MSYS2/MinGW on Linux/Windows target environment.
  - Operating Systems: Windows 10/11 for development; Linux distribution for deployment.
- Network:
  - Simulated network environment using local network and/or a cloud-based environment to mimic external connections.

6.0 Test Cases

Server Unit Tests and Performance:

Note: The test executable **test_server.exe** was attached without our submission on owl



```
Running Poll object correctly converts to JSON...
[TestInfo] Testing to_json(Poll) outputs expected structure and values.
[Expected] id='p123', question='What is your favorite color?', keys present: optionA, optionB, predi
ctionsA, votesB
[Received] {
  "description": "Choose wisely",
  "id": "p123",
  "optionA": "Red",
  "optionB": "Blue",
  "predictionsA": 0,
  "predictionsB": 0,
  "question": "What is your favorite color?",
  "revealed": false,
  "votesA": 0,
  "votesB": 0
}
[PASSED] Poll object correctly converts to JSON
Test duration: 0 ms
----------------------------------------

----------------------------------------
Running Generated Room ID is 4 characters...
[TestInfo] Testing if generateUniqueRoomID() returns 4 characters and no collision.
[Expected] Room ID length: 4
[Received] Room ID: "3tqj" (length = 4)
[PASSED] Generated Room ID is 4 characters
Test duration: 0 ms
----------------------------------------

----------------------------------------
Running Poll ID is unique within room...
[TestInfo] Testing generateUniquePollID() to avoid collision with existing poll 'AAAA'.
[Expected] New poll ID != 'AAAA' and length == 4
[Received] Poll ID: "b3VK"
[PASSED] Poll ID is unique within room
Test duration: 0 ms
----------------------------------------

----------------------------------------
[TestInfo] Testing UserVote to JSON serialization.
[Expected] votedA=true, predictedA=false
[Received] {
  "predictedA": false,
  "votedA": true
}
[PASSED] UserVote object correctly converts to JSON
Test duration: 0 ms
----------------------------------------

----------------------------------------
Running Error message JSON is formatted correctly...
[TestInfo] Testing buildErrorMessage() returns correct payload.
[Expected] type='error', error='sample error', retry=true
[Received] {
  "error": "sample error",
  "retry": true,
  "type": "error"
}
[PASSED] Error message JSON is formatted correctly
Test duration: 0 ms
----------------------------------------

----------------------------------------
Running WebSocket: Host receives proper response...
[TestInfo] Testing host session response from server.
[Expected] Response type: 'host', should contain 'hostingRoomCode'
[Received] {
  "hostingRoomCode": "k7kW",
  "polls": [],
  "type": "host"
}
[PASSED] WebSocket: Host receives proper response
Test duration: 574 ms
```

```
----------------------------------------

----------------------------------------
Running WebSocket: Participant joins existing room...
[TestInfo] Host created room with code: jH8K
[TestInfo] Testing participant session join response.
[Expected] type='participant', must contain keys: 'polls', 'votes'
[Received] {
  "polls": [],
  "type": "participant",
  "votes": {}
}
[PASSED] WebSocket: Participant joins existing room
Test duration: 124 ms
----------------------------------------

----------------------------------------
Running WebSocket: Threads clean up after disconnect...
[TestInfo] Verifying that server spawned thread for connection.
[Expected] Threads before stop > 0
[Received] Threads before stop: 1
[TestInfo] Verifying thread is cleaned up after disconnect.
[Expected] Threads after stop < Threads before
[Received] Threads after stop: 0
[PASSED] WebSocket: Threads clean up after disconnect
Test duration: 634 ms
----------------------------------------

----------------------------------------
Running Performance: Create 100 rooms quickly...
[TestInfo] Creating 100 separate host sessions (one room per host)...
[TestInfo] Expected 100 WebSockets, Received: 100
[PASSED] Performance: Create 100 rooms quickly
Test duration: 3177 ms
----------------------------------------

----------------------------------------
[TestInfo] Creating 200 polls in one room...
[DEBUG] Room code for bulk poll creation: Ce3Q
[TestInfo] Expected 200 polls, Received: 200
[PASSED] Performance: Create 200 polls in one room
Test duration: 903 ms
----------------------------------------

----------------------------------------
Running Performance: Handle 100 simultaneous votes...
[TestInfo] Simulating 100 participants voting on 1 poll...
[DEBUG] Room for voting: Uylv
[DEBUG] Poll ID to vote on: 6jLp
[TestInfo] Expected 100 votes recorded in poll.
[Received] votesA: 50, votesB: 50, total: 100
[Received] Individual vote records: 100
[PASSED] Performance: Handle 100 simultaneous votes
Test duration: 3931 ms
----------------------------------------

----------------------------------------
Running Performance: Spawn 100 WebSocket threads...
[TestInfo] Stress test: Opening 100 host WebSocket connections...
[TestInfo] Expected 100 connection threads. Got: 100
[PASSED] Performance: Spawn 100 WebSocket threads
Test duration: 4137 ms
----------------------------------------

----------------------------------------
Running Performance: 100 users join same room...
[TestInfo] Concurrent joins: Spawning 100 participants to join the same room...
[DEBUG] Created room code: KOD2
[TestInfo] Expected participant count (including host): 101 | Actual: 101
[PASSED] Performance: 100 users join same room
Test duration: 3692 ms
```

```
----------------------------------------

----------------------------------------
Running Performance: Broadcast poll to 100 users...
[TestInfo] Broadcasting 1 poll to 100 participants...
[PASSED] Performance: Broadcast poll to 100 users
Test duration: 3180 ms
----------------------------------------

----------------------------------------
Running Performance: Scale system to 10,000 users...
[TestInfo] Scaling test: Simulating 9999 users joining a room...
[DEBUG] Room code for 10k test: q5P8
[TestInfo] Expected users: 10000, Actual: 10000
[PASSED] Performance: Scale system to 10,000 users
Test duration: 8186 ms
----------------------------------------

----------------------------------------
Running Performance: Measure latency and throughput...
[TestInfo] Sending 500 poll creation requests to measure latency and throughput...
[DEBUG] Host room: NdcN
[Perf] Expected responses: 500
[Perf] Actual responses:   500
[Perf] Average Latency:    15.9988 ms
[Perf] Throughput:         62.5048 requests/sec
[PASSED] Performance: Measure latency and throughput
Test duration: 8035 ms
----------------------------------------

----------------------------------------
Running Performance: Monitor CPU and memory usage...
[TestInfo] Launching 300 clients for resource usage test with active voting...
[Perf] RAM Usage: 152 MB
[Perf] Threshold:  < 500 MB
[Perf] Process CPU Usage (vote spam burst): 13.3594 %
[PASSED] Performance: Monitor CPU and memory usage
Test duration: 16083 ms
----------------------------------------


========================================
Test Summary: 20 passed, 0 failed.
Total Duration: 52964 ms

☑ Passed Tests:
 - Random ID is 4 characters
 - Random IDs are mostly unique
 - Session ID is at least 16 characters
 - Poll object correctly converts to JSON
 - Generated Room ID is 4 characters
 - Poll ID is unique within room
 - UserVote object correctly converts to JSON
 - Error message JSON is formatted correctly
 - WebSocket: Host receives proper response
 - WebSocket: Participant joins existing room
 - WebSocket: Threads clean up after disconnect
 - Performance: Create 100 rooms quickly
 - Performance: Create 200 polls in one room
 - Performance: Handle 100 simultaneous votes
 - Performance: Spawn 100 WebSocket threads
 - Performance: 100 users join same room
 - Performance: Broadcast poll to 100 users
 - Performance: Scale system to 10,000 users
 - Performance: Measure latency and throughput
 - Performance: Monitor CPU and memory usage
========================================
```

Server Unit Test and Performance Test Explanation of Results:

- Random ID Generation Tests
    - Test: "Random ID is 4 characters"
        - What it tests:
            - Whether the generateRandomID() function produces an ID string with exactly 4 characters.
        - Expected Result:
            - ID length must be 4 (e.g., "NW4v").
        - Output:
            - The test logs the generated ID along with its length. For instance, it reports [DEBUG] ID: "NW4v" with length 4, and the test passes.
    - Test: "Random IDs are mostly unique"
        - What it tests:
            - The uniqueness of IDs generated over 1000 iterations, ensuring that at least 95% are unique to avoid collisions.
        - Expected Result:
            - At least 950 unique IDs out of 1000.
        - Output:
            - The log shows a unique count of 1000, confirming the randomness and reliability of the ID generator.

- Session and Poll Serialization Tests
    - Test: "Session ID is at least 16 characters"
        - What it tests:
            - That the generateUniqueSessionID() function produces a session ID with a minimum length of 16 characters.
        - Expected Result:
            - A session ID length ≥ 16, for example "jQj67fgfpwynpmib".
        - Output:
            - The test logs the generated session ID and verifies that its length meets the requirement.
    - Test: "Poll object correctly converts to JSON"
        - What it tests:
            - The correct serialization of a Poll object to JSON using the to_json() function.
        - Expected Result:
            - The resulting JSON must include the correct values (e.g., id: "p123", question: "What is your favorite color?") and contain all required keys such as optionA, optionB, predictionsA, and votesB.
        - Output:
            - The printed JSON matches expectations, verifying that the serialization is accurate.

- Unique Room and Poll ID Tests
    - Test: "Generated Room ID is 4 characters"
        - What it tests:
            - That the function generateUniqueRoomID() produces a valid room ID of length 4 without conflicting with existing IDs.
        - Expected Result:
            - Room ID length exactly 4 (e.g., "3tqj").
        - Output:
            - The test confirms that a new room ID of the correct length is generated.
    - Test: "Poll ID is unique within room"
        - What it tests:
            - That a new poll ID generated within a room does not match an existing poll ID (for example, it must not be "AAAA" if that value already exists) and is 4 characters long.
        - Expected Result:
            - A unique 4-character poll ID that is not equal to "AAAA".
        - Output:
            - The log shows the newly generated poll ID and confirms that it differs from the existing value.

- UserVote and Error Message Serialization Tests
    - Test: "UserVote object correctly converts to JSON"
        - What it tests:
            - The correct JSON serialization of a UserVote instance.
        - Expected Result:
            - The JSON should indicate votedA: true and predictedA: false.
        - Output:
            - The log displays the JSON output, and the values match the expectation.
    - Test: "Error message JSON is formatted correctly"
        - What it tests:
            - The output of the buildErrorMessage() function to ensure it properly formats an error message in JSON.
        - Expected Result:
            - The JSON should include "type": "error", "error": "sample error", and "retry": true.
        - Output:
            - The test logs the error JSON and all expected fields are present with correct values.

- WebSocket Integration Tests
    - Test: "WebSocket: Host receives proper response"
        - What it tests:
            - That when a host sends a host-hello message, the server responds

with a JSON message of type "host" that includes a hostingRoomCode.
    - Expected Result:
        - A response with type "host" containing a valid room code.
    - Output:
        - For example, the log shows the response with "hostingRoomCode": "k7kW", and the test confirms the field is present.
- Test: "WebSocket: Participant joins existing room"
    - What it tests:
        - That a participant can join a room created by a host and receives a JSON response of type "participant" containing the keys "polls" and "votes".
    - Expected Result:
        - The participant's response should include type "participant" and contain both polls and votes.
    - Output:
        - The logged output reflects that the response meets the criteria.

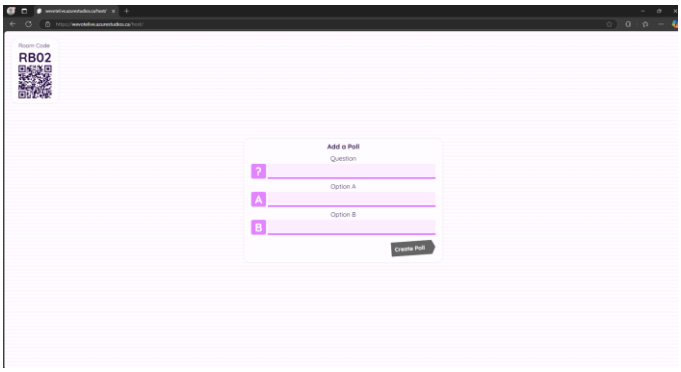
- Test: "WebSocket: Threads clean up after disconnect"
    - What it tests:
        - That after a WebSocket disconnect, the server cleans up its internal threads associated with that connection.
    - Expected Result:
        - The number of active connection threads should drop after a disconnect.
    - Output:
        - The test logs the number of threads before and after stopping, verifying that threads are removed.

- Performance Tests
    - Test: "Performance: Create 100 rooms quickly"
        - What it tests:
            - The server's ability to handle 100 simultaneous host connections, each creating its own room.
        - Expected Result:
            - Exactly 100 separate connection threads should be active.
        - Output:
            - Logs confirm 100 WebSockets were created, matching the expectation.
    - Test: "Performance: Create 200 polls in one room"
        - What it tests:
            - That the server can handle 200 poll creation requests in a single room.
        - Expected Result:
            - The room should end up with exactly 200 polls.

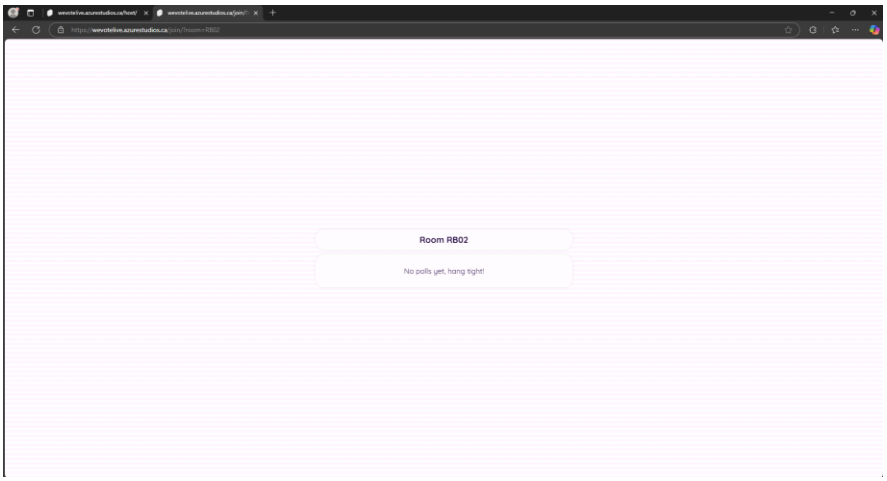
    - Output:
        - The log confirms that 200 polls were successfully created.
- Test: "Performance: Handle 100 simultaneous votes"
    - What it tests:
        - That 100 participants voting simultaneously on the same poll are all processed correctly.
    - Expected Result:
        - The total votes recorded should equal 100, with individual vote records matching.
    - Output:
        - The logs display 50 votes for each option, with total votes equal to 100.
- Test: "Performance: Spawn 100 WebSocket threads"
    - What it tests:
        - The server's ability to handle stress from 100 WebSocket connections spawned simultaneously.
    - Expected Result:
        - The number of active connection threads should be exactly 100.
    - Output:
        - The log confirms that 100 threads are active.
- Test: "Performance: 100 users join same room"
    - What it tests:
        - That 100 users (in addition to the host) can join the same room concurrently.
    - Expected Result:
        - The total participant count (including the host) should be 101.
    - Output:
        - The log indicates that 101 participants are in the room, matching the expectation.
- Test: "Performance: Broadcast poll to 100 users"
    - What it tests:
        - That when a poll is created, it is successfully broadcast to all 100 participant clients.
    - Expected Result:
        - Each participant should receive a message with exactly one poll.
    - Output:
        - Debug logs confirm that each received poll message has one question.
- Test: "Performance: Scale system to 10,000 users"
    - What it tests:
        - The server's ability to scale and accept nearly 10,000 users joining the same room in batches.
    - Expected Result:
        - The internal room participant map should reflect 10,000 users (including the host).

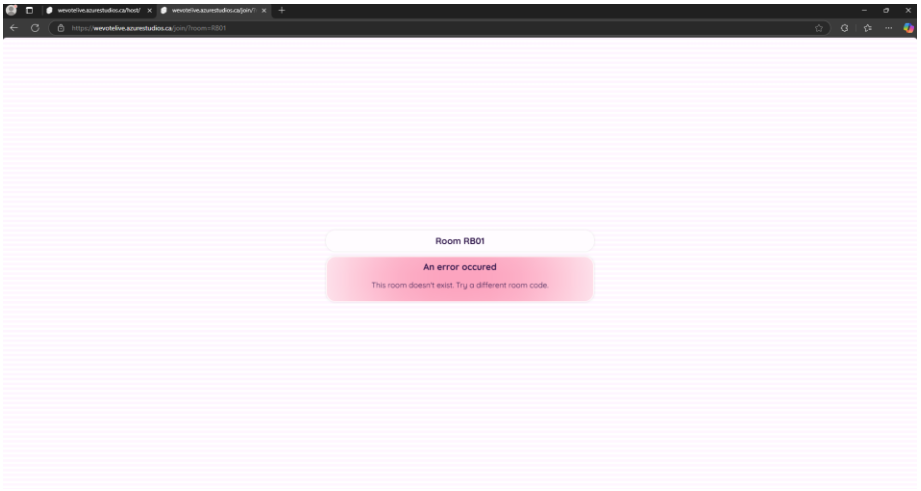
        - Output:
            - The log confirms that the total number of participants equals the expected total.
    - Test: "Performance: Measure latency and throughput"
        - What it tests:
            - The response times of the server when handling 500 poll creation requests, calculating the average latency and throughput (requests per second).
        - Expected Result:
            - A reasonable average latency (e.g., under 50 ms) and a throughput level above a certain threshold (e.g., > 50 req/sec).
        - Output:
            - The log displays measured average latency (e.g., ~16 ms) and throughput (e.g., ~62 req/sec), confirming performance objectives.
    - Test: "Performance: Monitor CPU and memory usage"
        - What it tests:
            - Under the load of 300 concurrent WebSocket clients that send voting traffic, the server's memory and CPU resource consumption remains within acceptable limits.
        - Expected Result:
            - Memory usage should be below an established threshold (< 500 MB), and CPU usage should be within acceptable ranges (< 30% usage).
        - Output:
            - The test logs show RAM usage (152 MB) and confirm CPU usage (13.35%), verifying that resource consumption is healthy.

Functional Tests

- Host Creates Room and Sees Code
    - Steps:
        - Go to the host page (/host).
        - Enter a name and click "Start Hosting".
    - Expected Output:
        - A unique 4-letter room code appears.
    - Output:
        - 

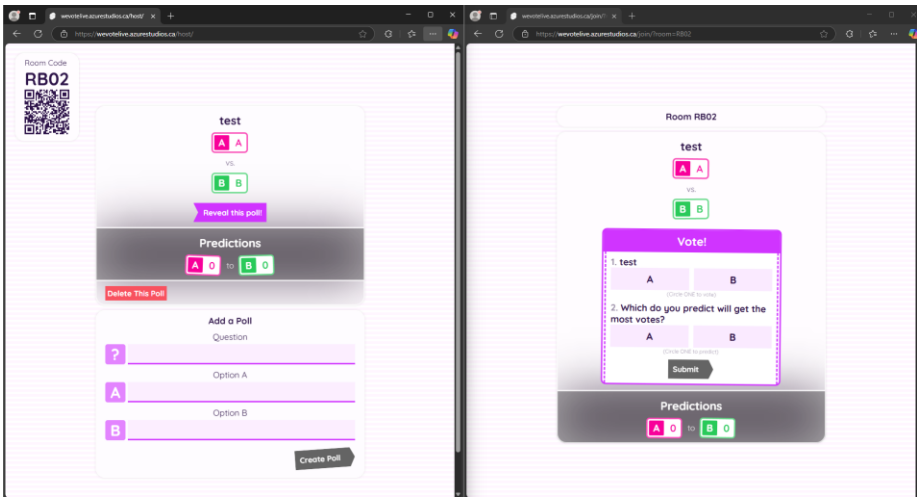
        - Passed! ✅

- Participant Enters Valid Room Code
    - Steps:
        - Open the participant page (/join).
        - Enter a name and valid room code (from a host).
    - Expected Output:
        - Page transitions to voting interface.
    - Output:
        - 

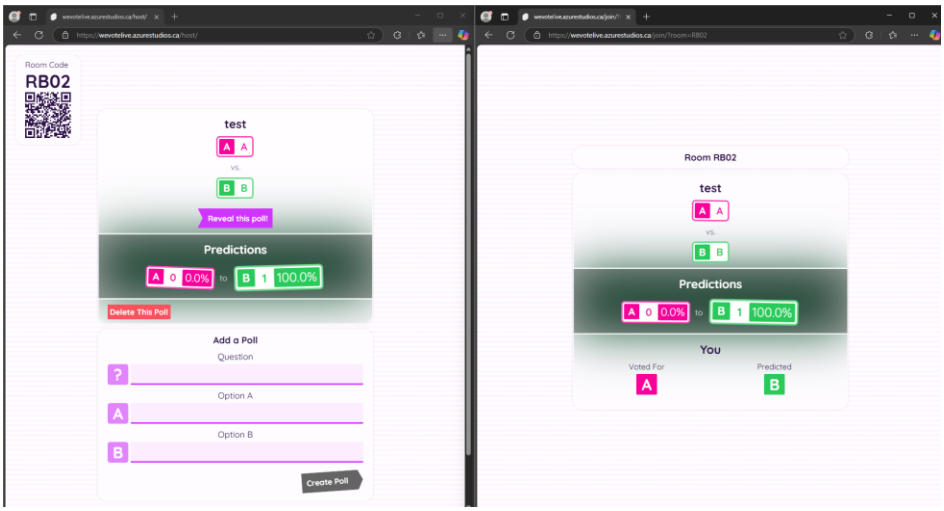
        - Passed! ✅
- Participant Enters Invalid Room Code
    - Steps:
        - Enter a random/invalid room code like “ZZZZ”.
    - Expected Output:
        - “the room doesn’t exist. Try a different room code” message shows
    - Output:
        - 

        - Passed! ✅

- Host Creates a Poll and It Appears
    - Steps:
        - From the host page, fill in a new poll (question + options).
        - Click “Create Poll”.

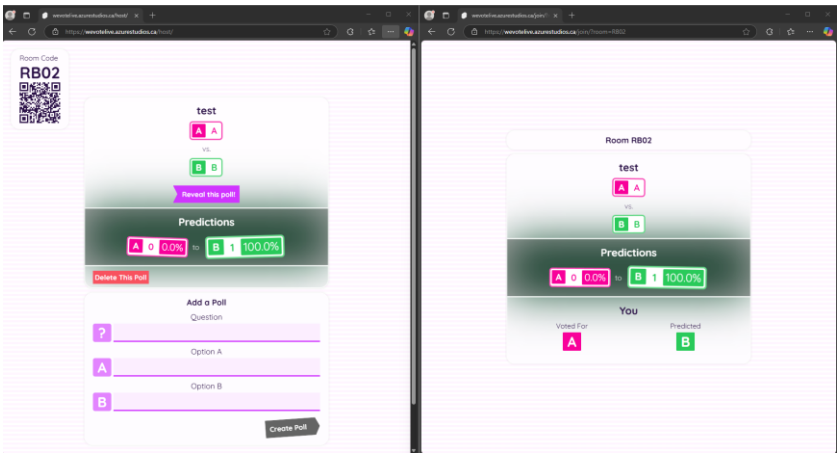
- Expected Output:
  - Poll appears in the list.
  - Participants see the new poll instantly.
- Output:
  - 

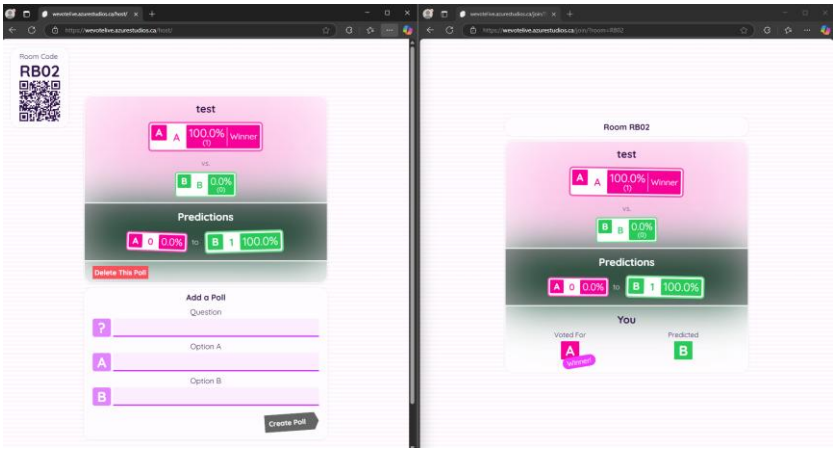
  - Passed! ✅

- Participant Votes on a Poll
  - Steps:
    - Join as participant.
    - Select an option for vote and prediction and clicks “Submit”.
  - Expected Output:
    - Button disables and UI confirms vote.
    - Host can see updated vote counts if revealed.
  - Output:
    - 

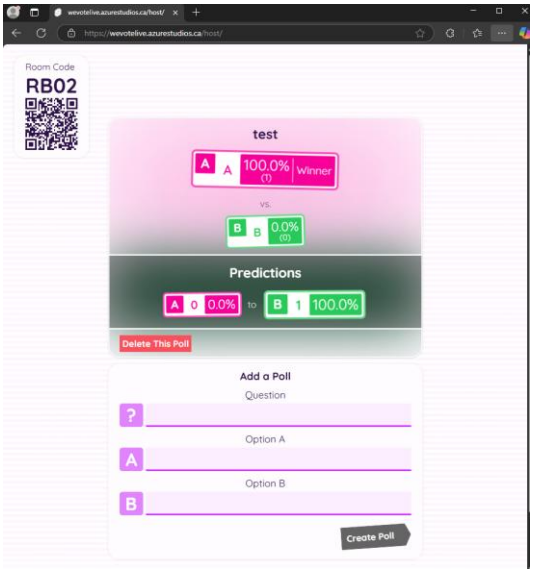
    - Passed! ✅

- Vote Prediction is Saved and Displayed
  - Steps:
    - Before voting, participant selects prediction (e.g., “I think A will win”).
  - Expected Output:
    - Prediction is sent to server and stored.

        - Host can later reveal results with prediction stats.
    - Output:
        - 

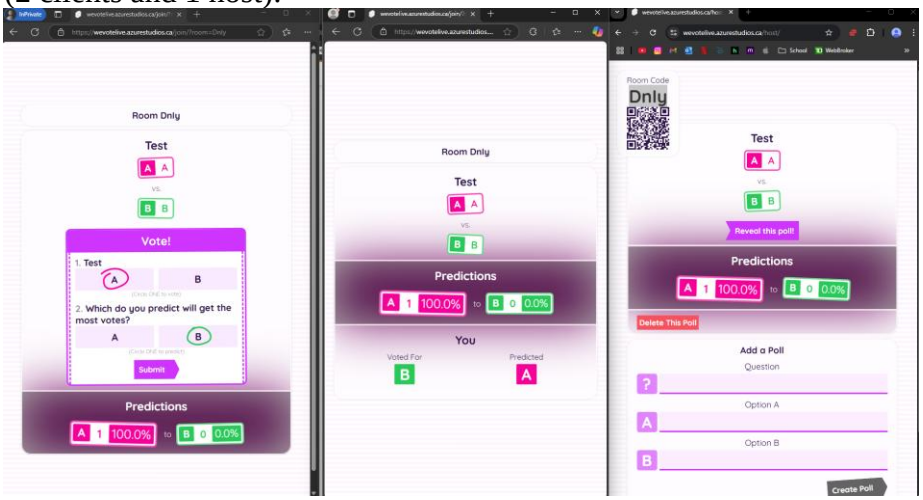
        - Passed! ✅
- Host Reveals Poll Results
    - Steps:
        - Click “Reveal” on an existing poll
    - Expected Output:
        - The percentage vote distribution appears.
        - Participant screens update to show results.
    - Output:
        - 

        - Passed! ✅
- Polls Persist in Host View
    - Steps:
        - Host creates multiple polls.
        - Refresh the page (but retain session state via cookies/localStorage)
    - Expected Output:
        - Previously created polls reload.
        - Room state is maintained if backend supports persistence.
    - Output: (post reloading)



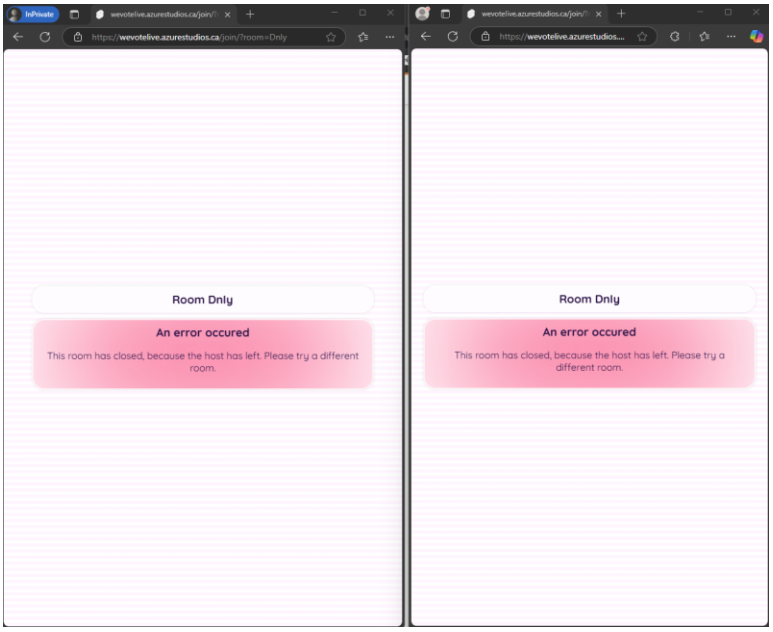
- Passed! ✅

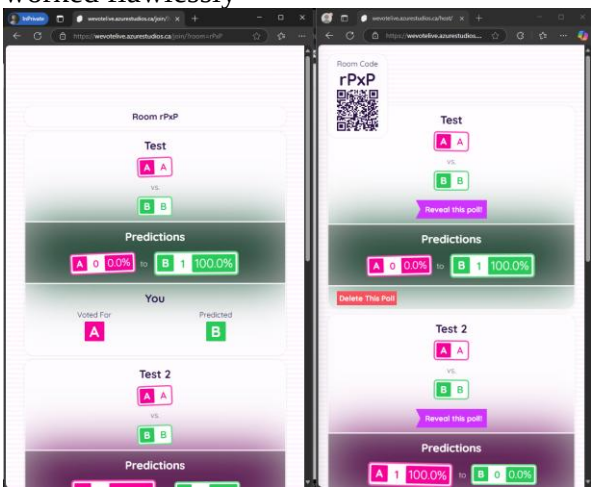
- Multiple Participants Join Same Room
  - Steps:
    - Open app in multiple tabs/devices. (note tabs hold cookies so you have to use different browsers on the same device)
    - Join same room with different names.
  - Expected Output:
    - All clients are active.
    - Polls broadcast to all clients correctly.
  - Output (2 clients and 1 host):



- Passed! ✅

- App Handles Disconnection Gracefully
  - Steps:
    - Host leaves for more than 1 minute
  - Expected Output:
    - Participants or notified that the room is closed
    - Server logs/threads should clean up disconnected clients.

- Output:
  - 

  - Passed! ✅
- App Handles Multiple Real Clients
  - Steps:
    - Host creates a room and creates 3 polls
    - 11 Participants Join and vote on all 3 polls concurrently
    - Host then reveals all polls
  - Expected Output:
    - Participants and Hosts should receive the proper updates (prediction updates, poll creations, reveals, etc) instantly
    - No latency or throughput changes compared to single participant single host scenario
  - Output:
    - While I couldn't screenshot multiple devices and put it here, in our test this worked flawlessly
    - 

    - Passed! ✅

Addendum: UI Testing

This addendum contains a table listing every major file that contributes to the UI, a brief description, and how it was tested.

File	Type	Description and Testing Method
Communication.svelte.ts	Class	Wraps a websocket, handling communication between the server and setting the world state. Tested by connecting to a local copy of the server to monitor proper events, inspecting the world states produced, and simulating edge cases.
Errors.ts	Dictionary	Contains a dictionary of error codes to error messages, used by the UI. Matched exhaustively to the list of possible error codes by hand.
Transitions.ts	Function	Contains the expandVertically transition PollBubbles use. Tested to avoid UI artifacts by playing it on all types of bubbles implemented.
types.d.ts	Type Definition	Contains types used to communicate between the server and client, such as the WorldState. Untested, as it is the source of truth for types.
title.svg	Asset	The logo. Tested to render correctly in common browsers by hand.
GenericSection.svelte	Component	Holds arbitrary section data in bubbles, used by pages. Used to correct layout. Tested during general page tests.
PollBubble.svelte	Component	Displays the card-like surface used by most UI elements. Uses the transition from Transitions.ts. Tested using special testing layouts and pages to refine parameters and stacking behavior.
PollButton.svelte	Component	Unused.
PollDivider.svelte	Component	Simple divider. Tested in layout with developer tools and zoom, rendering verified on various browsers during generic page testing.
PollHeader.svelte	Component	Displays the top information (question, stickers, winner, etc) of the Poll. Tested with fake poll data in a gallery, to ensure all situations render correctly and avoid regressions.
PollPredictionSegment.svelte	Component	Displays the prediction segment, similar to the header. Tested in combination with the header and divider, to ensure they both render correctly when used in the same card.
PollVoteCard.svelte	Component	Displays controls allowing the user to submit a vote. Tested with fake data and correct usage is evident when used against a real server instance. Transition (from Transitions.ts) also tested.
PollYouSection.svelte	Component	Displays the “You” section. Tested in a

		combination with all the other poll segments and with various fake data to simulate all possible outcomes.
SvgPenCircle.svelte	Component	Generates and animates a circle-scribble, used by PollVoteCard to highlight the selected option. Tested extensively with a grid of instances, to quickly see outcomes from the current randomness settings. Settings refined and grid refreshed until close enough to intended result.
global.css	Stylesheet	Tested amongst the rest of the components and pages during generic testing.
+layout.ts	Build Settings	Tested and verified by the SvelteKit Vite plugin during building. Causes additional behavior and verifications during building, along with setting up various export settings.
routes/+page.svelte	Page	The home page. Tested using development tools on various window sizes and browsers.
routes/host/+page.svelte	Page	The host page. Tested extensively in various configurations with a real server instance. Reactivity tested with development tools on various window sizes, and on various browsers.
routes/join /+page.svelte	Page	The Participant page. Tested extensively in various configurations with a real server instance. Also tested with various fake WorldData configurations, to simulate hard-to-achieve edge cases. Also used to test the composition, layout, and reactivity of most other Poll components using fake data. This includes thing such as unusually long words and phrases in various positions, unlikely outcomes, and strange viewport sizes. Tested on various browsers and viewport sizes.